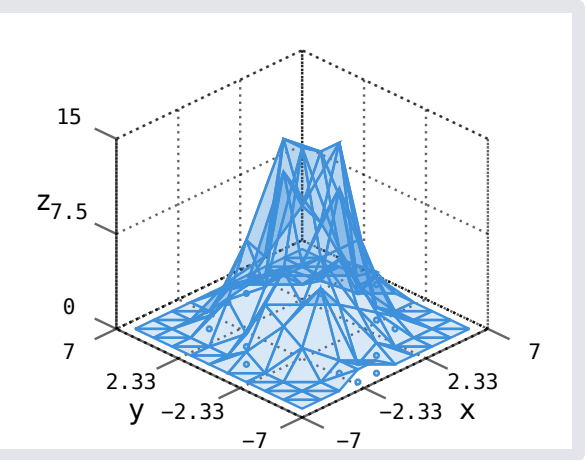
- distribution()
- line()
- lineplot()
- path()
- planeplot()
- polygon()
- quiver()
- vertices()

distribution

Histogram (distribution)

```
#let rng = suiji.gen-rng-f(26)
#let (rng, xs) = suiji.normal-f(rng, size:
200, scale: 2)
#let (rng, ys) = suiji.normal-f(rng, size:
200, scale: 2)
#pt.diagram(
  xaxis: (lim: (-7, 7)),
  yaxis: (lim: (-7, 7)),
  pt.distribution(
    xs, ys,
  )
)
```



Parameters

```
distribution(
  stroke: none auto length color gradient stroke tiling dictionary,
  fill: auto none color gradient tiling,
  label: none content,
  stroke-color-fn: none function,
  fill-color-fn: none function,
  mark: none mark,
  xn: int,
  yn: int,
  interpolate,
  xs: array,
  ys: array
)
```

stroke none or auto or length or color or gradient or stroke or tiling or dictionary

Stroke, defaults to the nth color-cycle entry

Default: auto

fill `auto` or `none` or `color` or `gradient` or `tiling`

Fill, defaults to the nth color-cycle entry

Default: `auto`

label `none` or `content`

Label

Default: `none`

stroke-color-fn `none` or `function`

Stroke color function, defaults to `distribution.stroke`

Default: `none`

fill-color-fn `none` or `function`

Fill color function, defaults to `distribution.fill`

Default: `none`

mark `none` or `mark`

Mark for outliers, one of `TODO: marks`

Default: `"."`

xn `int`

x axis sample size

Default: `10`

yn `int`

y axis sample size

Default: `10`

interpolate

TODO

Default: `none`

xs `array`

x data points

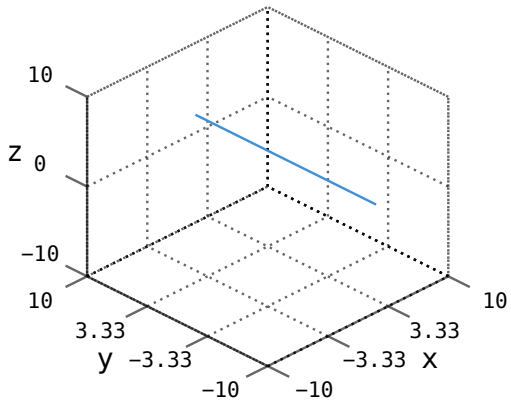
ys `array`

y data points

line

Renders line in point-normal form. Following helper functions exist for converting to point-normal:
TODO: line-parametric, line-point-normal, line-symmetric, line-points

```
#diagram(
  pt.line(
    ((2,0,2), (0,10,0))
  ),
)
```



Parameters

```
line(
  stroke: none auto length color gradient stroke tiling dictionary,
  label: none content,
  point-normal: array
)
```

stroke `none` or `auto` or `length` or `color` or `gradient` or `stroke` or `tiling` or `dictionary`

Stroke, defaults to the nth color-cycle entry

Default: `auto`

label `none` or `content`

Label

Default: `none`

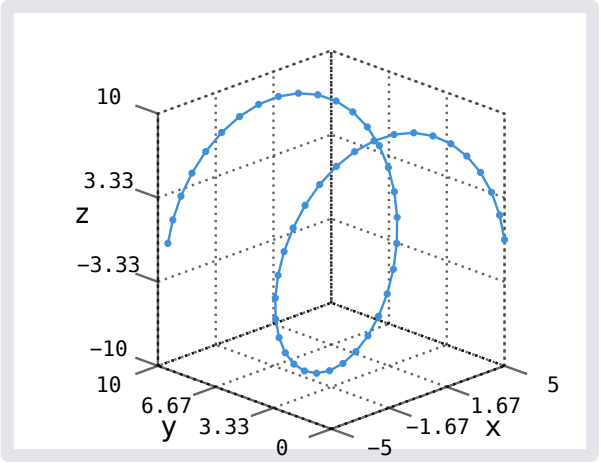
point-normal `array`

Point-normal form of the line (p, d)

lineplot

Plotted line

```
#let xs = pt.linspace(0,calc.pi * 3)
#pt.diagram(
   pt.lineplot(
     xs.map(x => calc.cos(x) * 5),
     xs,
     xs.map(x => calc.sin(x) * 10),
   )
 )
```



Parameters

```
lineplot(
  stroke: none auto length color gradient stroke tiling dictionary,
  label: none content,
  stroke-color-fn: none function,
  mark: none mark,
  xs: array,
  ys: array,
  zs: array
)
```

stroke `none` or `auto` or `length` or `color` or `gradient` or `stroke` or `tiling` or `dictionary`

Stroke, defaults to the nth color-cycle entry

Default: `auto`

label `none` or `content`

Label

Default: `none`

stroke-color-fn `none` or `function`

Stroke color function, defaults to `lineplot.stroke`

Default: `none`

mark `none` or `mark`

Mark for intermediary points, one of `TODO: marks`

Default: `"."`

xs `array`

x data points

ys `array`

y data points

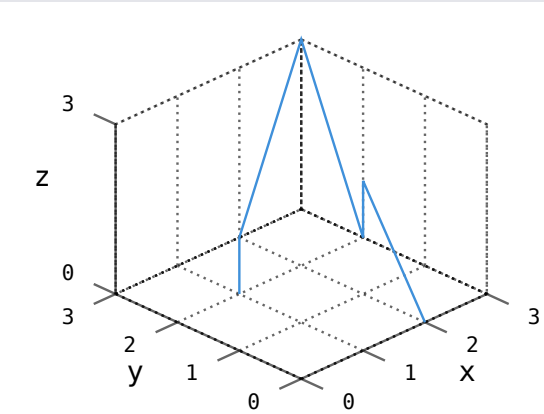
zs `array`

z data points

path

Path along points

```
#diagram(
  pt.path(
    (0,1,1), (1,2,1), (3,3,3),
    (3,2,0), (3,2,1), (2,0,0)
  ),
)
```



Parameters

```
path(
  stroke: none auto length color gradient stroke tiling dictionary,
  label: none content,
  stroke-color-fn: none function,
  mark: none mark,
  ..points: array
)
```

stroke `none` or `auto` or `length` or `color` or `gradient` or `stroke` or `tiling` or `dictionary`

Stroke, defaults to the nth color-cycle entry

Default: `auto`

label `none` or `content`

Label

Default: `none`

stroke-color-fn `none` or `function`

Stroke color function, defaults to `path.stroke`

Default: `none`

mark `none` or `mark`

Mark for intermediary points, one of `TODO: marks`

Default: `none`

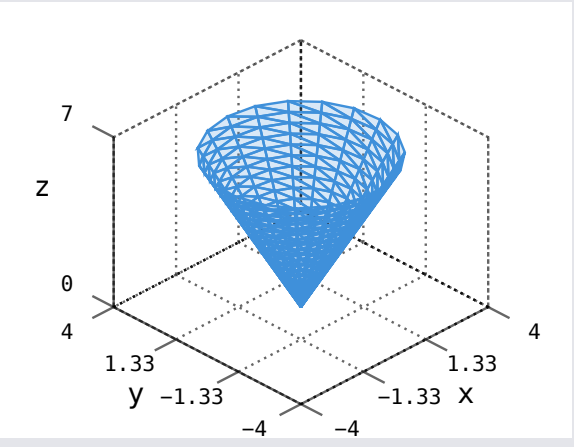
..points `array`

Points as (x, y, z)

planeplot

Plotted surface

```
#let num = 20
#let domain = pt.domain((0, calc.pi), (0,
2 * calc.pi), v-num: num, u-num: num)
#diagram(
  pt.planeplot(
    domain.map(((u, v)) => u *
calc.sin(v)),
    domain.map(((u, v)) => u *
calc.cos(v)),
    domain.map(((u, v)) => u * 2),
    num: num,
  ),
)
```



Parameters

```
planeplot(
  stroke: none auto length color gradient stroke tiling dictionary,
  fill: auto none color gradient tiling,
  label: none content,
  stroke-color-fn: none function,
  fill-color-fn: none function,
  num: none int,
  xs: array,
  ys: array,
  zs: array
)
```

stroke `none` or `auto` or `length` or `color` or `gradient` or `stroke` or `tiling` or `dictionary`

Stroke, defaults to the nth color-cycle entry

Default: `auto`

fill `auto` or `none` or `color` or `gradient` or `tiling`

Fill, defaults to the nth color-cycle entry

Default: `auto`

label `none` or `content`

Label

Default: `none`

stroke-color-fn `none` or `function`

Stroke color function, defaults to `planeplot.stroke`

Default: `none`

fill-color-fn `none` or `function`

Fill color function, defaults to `planeplot.fill`

Default: `none`

num `none` or `int`

TODO:

Default: `none`

xs `array`

x data points

ys `array`

y data points

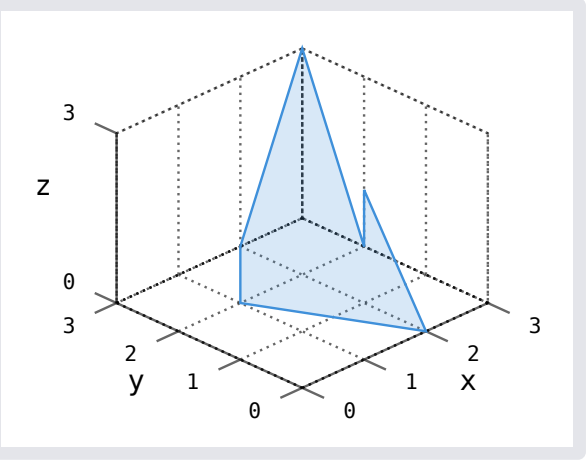
zs `array`

z data points

polygon

Polygon along points

```
#diagram(
  pt.polygon(
    (0,1,1), (1,2,1), (3,3,3),
    (3,2,0), (3,2,1), (2,0,0)
  ),
)
```



Parameters

```
polygon(
  stroke: none auto length color gradient stroke tiling dictionary,
  fill: auto none color gradient tiling,
  label: none content,
  ..points: array
)
```

stroke `none` or `auto` or `length` or `color` or `gradient` or `stroke` or `tiling` or `dictionary`

Stroke, defaults to the nth color-cycle entry

Default: `auto`

fill `auto` or `none` or `color` or `gradient` or `tiling`

Fill, defaults to the nth color-cycle entry

Default: `auto`

label `none` or `content`

Label

Default: `none`

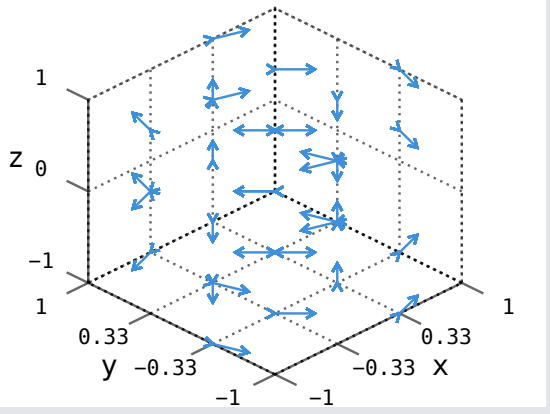
..points `array`

Points as (x, y, z)

quiver

Quiver diagram (vector field)

```
#let xs = pt.linspace(-1, 1, num: 4)
#let (xs, ys, zs) = pt.meshgrid(xs, xs,
xs, pts: false)
#pt.diagram(
  pt.quiver(
    xs, ys, zs,
    (x, y, z) => (y / z, -x / z, 0),
    norm: true,
    scale: .3,
  )
)
```



Parameters

```
quiver(
  stroke: none auto length color gradient stroke tiling dictionary,
  fill: auto none color gradient tiling,
  label: none content,
  stroke-color-fn: none function,
  fill-color-fn: none function,
  tip: tip,
  toe: tip,
  scale: int float,
  norm: bool,
  xs: array,
  ys: array,
  zs: array,
  dir: function
)
```

stroke `none` or `auto` or `length` or `color` or `gradient` or `stroke` or `tiling` or `dictionary`

Stroke, defaults to the nth color-cycle entry

Default: `auto`

fill `auto` or `none` or `color` or `gradient` or `tiling`

Fill, defaults to the nth color-cycle entry

Default: `auto`

label `none` or `content`

Label

Default: `none`

stroke-color-fn `none` or `function`

Stroke color function, defaults to `quiver.stroke`

Default: `none`

fill-color-fn `none` or `function`

Fill color function, defaults to `quiver.fill`

Default: `none`

tip `tip`

Vector tip, one of `TODO: tips`

Default: `">"`

toe `tip`

Vector toe, one of `TODO: tips`

Default: `none`

scale `int` or `float`

Scale

Default: `1`

norm `bool`

If vectors should be normalized

Default: `false`

xs `array`

x data points

ys `array`

y data points

zs `array`

z data points

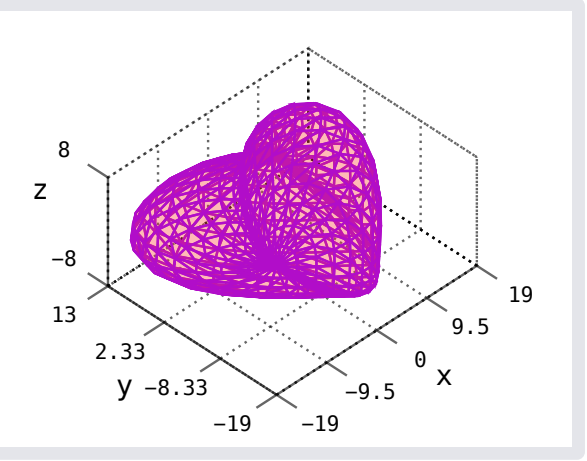
dir `function`

Direction function

vertices

Vertices

```
#pt.diagram(
  pt.vertices(
    fill: red.transparentize(80%),
    stroke: purple,
    ..pt.heart()
  )
)
```



Parameters

```
vertices(
  stroke: none auto length color gradient stroke tiling dictionary,
  fill: auto none color gradient tiling,
  label: none content,
  stroke-color-fn: none function,
  fill-color-fn: none function,
  ..vertices: array
)
```

stroke `none` or `auto` or `length` or `color` or `gradient` or `stroke` or `tiling` or `dictionary`

Stroke, defaults to the nth color-cycle entry

Default: `auto`

fill `auto` or `none` or `color` or `gradient` or `tiling`

Fill, defaults to the nth color-cycle entry

Default: `auto`

label `none` or `content`

Label

Default: `none`

stroke-color-fn `none` or `function`

Stroke color function, defaults to `vertices.stroke`

Default: `none`

fill-color-fn `none` or `function`

Fill color function, defaults to `vertices.fill`

Default: `none`

..vertices `array`

Vertices as (x, y, z)

- apply-matrices()
- cross-product()
- direction-vec()
- distance-vec()
- distance-vec-squared()
- dot-product()
- length-vec()
- line-parametric()
- line-points()
- line-symmetric()
- mat-mult-vec()
- mat-rotate-x()
- mat-rotate-y()
- mat-rotate-z()
- normalize-vec()
- plane-coordinate()
- plane-hesse()
- plane-parametric()
- plane-point-normal()
- plane-points()

- sum-vec()

Variables
- plane-normal
- line-point-normal
- mat-rotate-iso

apply-matrices
Multiplies a $\mathbb{R}^3$ vector with $\mathbb{R}^{3\times3}$ matrices

Parameters
```
apply-matrices(
  v: vector,
  ..m: matrices
) -> matrix
```

v `vector`

Vector

..m `matrices`

Matrices

cross-product
Calculates the cross product of two vectors

```
#cross-product((1,3,2), (2,0,1))
```

```
(3, 3, -6)
```

Parameters
```
cross-product(
  (x1, y1, z1): vector,
  (x2, y2, z2): vector
) -> vector
```

(x1, y1, z1) `vector`

w

(x2, y2, z2) `vector`

v

direction-vec
Calculates the direction vector of two vectors

```
#direction-vec((0,1,2), (2,0,1))
```

```
(2, -1, -1)
```

Parameters

```
direction-vec(
  from: vector,
  to: vector
) -> vector
```

from `vector`

Start vector

to `vector`

End vector

distance-vec

Calculates the distance between two vectors

```
#distance-vec((0,1,2), (2,0,1))
```

```
2.449489742783178
```

Parameters

```
distance-vec(
  from: vector,
  to: vector
) -> vector
```

from `vector`

Start vector

to `vector`

End vector

distance-vec-squared

Calculates the squared distance between two vectors

```
#distance-vec-squared((0,1,2), (2,0,1))
```

```
6
```

Parameters

```
distance-vec-squared(
  from: vector,
  to: vector
) -> vector
```

from `vector`

Start vector

to `vector`

End vector

dot-product

Calculates the dot product of two vectors

```
#dot-product((1,3,2), (2,0,1))
```

```
4
```

Parameters

```
dot-product(
  v: vector,
  w: vector
) -> vector
```

v `vector`

v

w `vector`

w

length-vec

Calculates the length of a vector

```
#length-vec((0,3,4))
```

```
5
```

Parameters

```
length-vec(v: vector) -> vector
```

v `vector`

Vector

line-parametric
Constructs line from parametric form

Parameters

```
line-parametric(
  p: vector,
  d: vector
) -> line
```

p `vector`

Point on line

d `vector`

Direction vector

line-points
Constructs line from given points

Parameters

```
line-points(
  a: vector,
  b: vector
) -> line
```

a `vector`

Point 1

b `vector`

Point 2

line-symmetric
Constructs line from symmetric form

Parameters

```
line-symmetric(
  x: int float,
  dx: int float,
  y: int float,
  dy: int float,
  z: int float,
  dz: int float
) -> line
```

x `int` or `float`

x

dx `int` or `float`

x-intercept

y `int` or `float`

y

dy `int` or `float`

y-intercept

z `int` or `float`

z

dz `int` or `float`

z-intercept

mat-mult-vec

Multiplies a $\mathbb{R}^{3\times3}$ matrix with a $\mathbb{R}^3$ vector

```
#mat-mult-vec((
    (1,0,0),
    (0,1,0),
    (0,0,1)
  ), (2,0,1))
```

```
(2, 0, 1)
```

Parameters

```
mat-mult-vec(
  ((a, b, c), (d, e, f), (g, h, i)): matrix,
  (x, y, z): vector
) -> vector
```

((a, b, c), (d, e, f), (g, h, i)) `matrix`

Matrix

(x, y, z) `vector`

Vector

mat-rotate-x

Constructs a rotation matrix in the x direction

Parameters

```
mat-rotate-x(x: int float) -> matrix
```

x `int` or `float`

Amount to rotate by

mat-rotate-y

Constructs a rotation matrix in the y direction

Parameters

```
mat-rotate-y(y: int float) -> matrix
```

y `int` or `float`

Amount to rotate by

mat-rotate-z

Constructs a rotation matrix in the y direction

Parameters

```
mat-rotate-z(z: int float) -> matrix
```

z `int` or `float`

Amount to rotate by

normalize-vec

Normalizes a vector

```
#normalize-vec((0,3,4))
```

```
(0.0, 0.6, 0.8)
```

Parameters

```
normalize-vec(v: vector) -> vector
```

v `vector`

Vector

plane-coordinate

Constructs plane from coordinate form

Parameters

```
plane-coordinate(
  x: int float,
  y: int float,
  z: int float,
  d: int float
) -> plane
```

x `int` or `float`

x

y `int` or `float`

y

z `int` or `float`

z

d `int` or `float`

Distance

plane-hesse

Constructs plane from hesse form

Parameters

```
plane-hesse(
  (x, y, z): vector,
  d: int float
) -> plane
```

(x, y, z) `vector`

Normal vector

d `int` or `float`

Distance

plane-parametric

Constructs plane from parametric form

Parameters

```
plane-parametric(
  p: vector,
  v: vector,
  w: vector
) -> plane
```

p `vector`

Point on plane

v `vector`

Non-colinear vector 1

w `vector`

Non-colinear vector 2

plane-point-normal

Constructs plane from point normal form

Parameters

```
plane-point-normal(
  n: vector,
  p: vector
) -> plane
```

n `vector`

Normal vector

p `vector`

Point on plane

plane-points

Constructs plane from given points

Parameters

```
plane-points(
  a: vector,
  b: vector,
  c: vector
) -> plane
```

a `vector`

Point 1

b `vector`

Point 2

c `vector`

Point 3

sum-vec

Adds an arbitrary amount of vectors

```
#sum-vec((0,1,2), (2,0,1), (-1, 0, 1))
```

```
(1, 1, 4)
```

Parameters

```
sum-vec(..v: vectors) -> vector
```

..v `vectors`

The vectors to sum up

plane-normal `plane`

Constructs plane from normal form (alias for `plane-hesse`)

line-point-normal `line`

Constructs line from point-normal form (alias for `line-parametric`)

mat-rotate-iso `matrix`

Isometric rotation matrix